
A Unified Framework of Concepts and Chunks

Karthik Singaravadivelan

KSINGARA3@GATECH.EDU

Pat Langley

PATRICK.W.LANGLEY@GMAIL.COM

Institute for the Study of Learning and Expertise, 2164 Staunton Court, Palo Alto, CA 94306 USA

Abstract

Humans routinely categorize the observations they encounter and combine familiar parts into novel composites. The cognitive-systems literature has long studied these two abilities in parallel, one under the heading of concept formation and the other under the heading of chunking. Neither line of work, taken alone, accounts for the joint exercise that grammatical awareness demands. In this paper, we present a postulate-driven theory that treats concept-hood and chunk-hood as two aspects of every representation. The theory builds on earlier concept-formation work in the Cobweb family and extends it in new directions, adding a compositional vocabulary and matched processes for building and unpacking composites. We also describe TRELLIS, an implemented system that instantiates the theory, and demonstrate its behavior on a series of synthetic context-free grammars. We conclude by discussing related approaches to concepts and chunks and directions for future research.

1. Introduction

The cognitive-systems literature has long sought computational accounts of how people organize experience into recognizable structure. Two lines of work developed in parallel (Langley, 2025): *concept formation* groups an observed instance with categories whose members it has never appeared with; *chunking* combines familiar parts into novel composites. Both are on display when a listener processes a sentence: individual words are categorized by part of speech and successive constituents are combined into a partonomic structure. We adopt grammar induction as the empirical domain because it exercises the two abilities together, and because gold parse trees provide an unambiguous target.

Concepts are categorical, graded mental structures that summarize regularities shared by a set of entities and are organized taxonomically (Rosch et al., 1976; Murphy, 2002). The Cobweb family (Fisher, 1987; Gennari et al., 1989), which we build on, incrementally sorts attribute–value instances into such a taxonomy. Every variant inherits the same flat substrate: an instance is a set of unrelated pairs, and the vocabulary contains no name for how the pairs relate. Compositional structure thus sits outside what any concept-formation system in this lineage can express, and psychological work on how concepts combine (Osherson & Smith, 1981; Hampton, 1988; Murphy & Medin, 1985) shows the gap is substantive.

Chunks, by contrast, denote collections of entities in a specified configuration (Miller, 1956; Chase & Simon, 1973). Chess masters reproduce briefly-glimpsed configurations far better than amateurs because they perceive the board in familiar multi-piece chunks (Chase & Simon, 1973),

and language learners aggregate phonemes into syllables and syllables into words as fluency grows. The idea was formalized in CHREST (Gobet & Lane, 2012), a descendant of EPAM (Richman, 1991), and runs through much of the grammar-induction literature (Wolff, 1982; Langley & Stromsten, 2000). But chunking systems share a limitation of their own: a chunk records a specific configuration of parts, so it cannot on its own recognize a structurally identical composite whose component words it has never seen. Compositionality is present; categorical generalization is not.

We take the position that a unified account should treat concept-hood and chunk-hood as two aspects of every representation, and adopt Cobweb as our starting point because it already delivers the categorical half. What it lacks is a vocabulary for the parts that compose an element and matched processes for composing children into recognized elements and decomposing them back. Every element in our theory carries a *content* description that names its parts and a *context* description that names its surroundings; two hierarchies index the same elements through these complementary lenses. A chunk, in this view, is simply a composite concept—a Cobweb category whose instances are themselves composed elements—so chunk-hood and concept-hood become two descriptions of one representation rather than two kinds of thing.

We make three contributions. First, we develop a postulate-driven unified theory of concepts and chunks that extends Cobweb. Second, we describe a concrete system that unifies parsing, generation, and learning under a single elaboration loop. Third, we report an empirical study on synthetic grammar-induction tasks characterizing how each hierarchy contributes to categorical and compositional accuracy. We review Cobweb in Section 2, present the extending postulates in Section 3, describe the implemented system in Section 4, report the evaluation in Section 5, and situate the work against related literature and open questions in Sections 6 and 7.

2. A Review of Cobweb

Fisher (1987)’s original Cobweb introduced an incremental, unsupervised taxonomic learner over discrete attribute–value instances. Later variants extended the framework: TRESTLE (MacLellan et al., 2016) handled relational instances; Cobweb/4L (MacLellan et al., 2022) refined the basic-level computation; convolutional variants (Lian & MacLellan, 2024) extended reach to perceptual data; the Matsakis system (Matsakis et al., 2023) showed that carefully crafting the representation can adapt the substrate to a new domain, an observation Section 4 leans on. Every variant shares the same four commitments, which we restate below as postulates so the chunking theory of Section 3 can extend them directly.

2.1 Representation

Cobweb draws a sharp line between the raw units of experience it takes in and the categorical summaries it builds from them.

- R1.** Knowledge takes two forms: instances and concepts.
- R2.** An instance is a set of attribute–value pairs describing a single observation.
- R3.** A concept is a probabilistic summary of a set of instances, expressed as counts over attribute–value pairs.

Concepts in Cobweb hold many instances as aggregated counts and correspond to categorical generalizations, not exemplars: each concept records, per attribute, a tally of how often each value was observed among the instances routed through it. Cobweb treats attributes as conditionally independent given a concept, with each value distribution smoothed by a uniform prior. This assumption is what lets us route an element’s content and context descriptions through two separate hierarchies in Section 3 without changing what happens at any node. The representational vocabulary matters as much as the sorting mechanism, an observation Matsakis et al. (2023) made concrete by showing that carefully crafting a phrase-structure representation was enough to adapt Cobweb to grammar induction; our theory generalizes this observation into a permanent representational commitment.

2.2 Organization

Concepts are not stored in isolation; Cobweb arranges them in a structure that supports classification.

- 01.** Concepts are organized in a taxonomic hierarchy rooted at a single concept that summarizes all observed instances.
- 02.** Each non-terminal node has two or more children that exclusively partition the instances it summarizes, and instances themselves reside as the terminal nodes.

Each non-root node refers to a strict subset of its parent’s instances, so descent is specialization rather than composition. Terminal nodes summarize singleton instances, so the same routing machinery handles intermediate abstractions and individual observations. The taxonomic structure is what our theory preserves while adding a partonomic dimension on top.

2.3 Performance

The organization above supports two performance routines that both run over the same hierarchy: recognition, which places a new instance, and prediction, which reads off unobserved attributes from the destination.

- P1.** An instance is sorted down the hierarchy from the root toward a terminal node, with each non-terminal routing the instance to the child concept that best fits it under a sorting criterion.
- P2.** Predictions about unobserved attributes of an instance are read from the concept to which that instance is sorted.

Cobweb’s descent is single-pass: it commits to one child at each level and never backtracks.¹ The sorting criterion is *category utility* (Fisher, 1987), a probabilistic score that rewards a candidate concept when its instances share attribute values (intra-category coherence) and differ from those of sibling concepts (inter-category discriminability).

2.4 Learning

Learning in Cobweb is not a separate offline stage but an update rule attached to the same sorting descent that supports recognition.

1. Cobweb/4V replaces this greedy descent with a best-first search under an information-theoretic objective; we retain the greedy version because it interleaves with the parsing loop of Section 4 without additional bookkeeping.

- L1.** Learning is interleaved with performance: each sorted instance is absorbed into the hierarchy along the same path that classified it.
- L2.** At each non-terminal along the path, the system selects among four actions: *add* the instance to an existing child, *create* a new child, or *restructure* the hierarchy via *merge* or *split*.

The four-action menu separates two concerns: *add* and *create* grow the tree without rearranging existing structure, while *merge* and *split* restructure it. The action selected at every step improves the same criterion that routed the instance.

R1–L2 fix Cobweb as an incremental concept-formation substrate: powerful along the categorical dimension but silent along the compositional one, because its representational vocabulary contains no name for how the pairs inside an instance relate. Section 3 supplies those missing pieces.

3. A Unified Theory of Chunking and Categorization

The Cobweb commitments of Section 2 carry directly into the unified theory we propose, and the postulates below extend them with numbering continued from Section 2 to make the inheritance explicit. Cobweb’s substrate is missing two things to accommodate compositional elements: a vocabulary for the parts that compose an element, and matched processes for composing children into recognized elements and decomposing them back. Recent Cobweb variants have moved in complementary directions; the Cobweb/4L family (MacLellan et al., 2022) recasts the sorting criterion under an information-theoretic objective, whereas our extension holds the objective fixed and enriches what each instance can express. The postulates lay out the enriched vocabulary, the two hierarchies the theory posits, and the composition and decomposition processes; we use grammatical structure throughout as a running example because it exercises the categorical and compositional dimensions in tandem, though the postulates apply to any domain whose experiences carry discrete relational structure.

3.1 Representation

The theory’s representational vocabulary starts from the same instance/concept distinction as Cobweb but broadens what an experience can contain, and introduces the compositional element as the unit that memory organizes.

- R4.** An experience is a collection of instances together with a set of discrete relations between them, represented as a set of primitive elements and the local relations that arrange them in the experience.
- R5.** Every element carries two descriptions: a *content* description, which names the parts that compose it, and a *context* description, which names the discrete relations it bears to surrounding elements.
- R6.** Every element is either a primitive or a composite; a composite (also called a *chunk*) is an element built from other primitive or composite elements, and its content description names those parts in the same vocabulary the rest of the system uses.

The two descriptions let the same element be recognized through two independent categorical lenses. Content bears on composition: a composite names its children, and a primitive names nothing. Context bears on distribution: for a primitive, a small sliding window of the tokens immediately to its left and right in the surface sequence; for a composite, the same window taken over its outermost positions (so the two boundaries of the composite are what neighbors see). A single attribute–value bundle would force role and filler into one signal, whereas two indexed descriptions keep them from interfering. This closure is deliberate and necessary: because a composition step must name its result in the same vocabulary as the primitives and composites it sits among, that result is itself a legal input to later composition, so a single recognition-and-composition mechanism applies unchanged at every depth of the partonomy and a finite vocabulary yields an unbounded space of composites.

3.2 Organization

Given elements that each carry two descriptions, memory must record both what an element was built out of and how each of its descriptions lands in category space.

- O3.** Primitives and composites are jointly organized in a partonomic hierarchy that records, for each composite, the elements it was built from.
- O4.** The system maintains long-term memory in the form of two distinct taxonomic hierarchies: a *content hierarchy* that organizes the content descriptions of R5, and a *context hierarchy* that organizes the context descriptions of R5.

The partonomic hierarchy of O3 holds a single parse together and answers “what are the parts?”. The two taxonomic hierarchies of O4 divide the categorical work. The *content hierarchy* stores *how elements combine*: organizing composites by the parts they are built from, each path records which children recur together as a well-formed composition. The *context hierarchy* does the reciprocal work of *representation*: sorting a composite by its surroundings crafts a new identifier for it in the same representational space as the primitives and composites around it, capturing how the composition behaves relative to elements already seen and letting later composites name it as a part. The two are in practice entangled, but keeping them separate is what lets the system both store how elements are composed and represent those compositions in a shared space so they can be composed in turn.

3.3 Performance

Performance runs in two directions: parsing builds composites from primitives; generation expands them back. The two subsections that follow share the same template (an incremental step, a candidate set, a decision rule, a termination condition).

Parsing.

Parsing runs bottom-up: it proposes composites over the elements currently on the frontier, commits the most recognizable one, and grows a partonomic tree until no further composite is recognized.

- P3.** Parsing is an incremental process of elaborations on an input experience, and each elaboration generates a set of candidate chunks consistent with the current partial parse.

P4. Each candidate chunk is scored against the content and context hierarchies and gated by a recognition threshold; the best-scoring candidate that passes the threshold is committed to the partonomic parse tree as a new composite element.

P5. Parsing terminates when no remaining candidate passes the recognition threshold.

The content score asks whether the composition looks like one the system has consolidated before; the context score asks whether the surroundings look like ones that have hosted similar composites. P5 lets the system fail gracefully: when nothing clears the threshold, the parse ends rather than assert spurious chunks at the boundary of competence.

Generation.

Generation runs the same loop in reverse: it starts from a recognized seed and expands it into children until only primitives remain.

P6. Generation is an incremental process of expansions on a composite seed, and each expansion produces a candidate leaf consistent with the basic-level concept above the seed’s content reference.

P7. The candidate leaf is sampled from the content hierarchy and conditioned on the context hierarchy; the sampled leaf is committed to the partonomic generation tree as the seed’s expansion, and its content children (if any) become new seeds.

P8. Generation terminates when every expanded element is a primitive.

The *basic-level* constraint of P6 plays the same role for generation that the recognition threshold plays for parsing. Basic-level concepts, in Rosch’s original sense (Rosch et al., 1976), are the level of abstraction that people access first when they name or reason about a category (“dog” rather than “mammal” or “poodle”); operationally, it is the shallowest node in the hierarchy that has already accumulated enough support to summarize a recurring configuration. Anchoring generation at such a node keeps sampling from committing to an idiosyncratic deep leaf, and conditioning the sampled leaf on context (P7) keeps the output distributionally consistent with what the system has already seen.

3.4 Learning

Learning in our theory is not a separate mechanism attached to parsing or generation. It is the observation that Cobweb’s sort-and-learn cycle already knows how to absorb any structured experience, whether that experience arrived as an already-structured input or was assembled during parsing. Whenever a structured experience is available, its content and context instances can be committed to the two hierarchies exactly the way Cobweb’s cycle commits any ordinary instance.

L3. Learning consists of incorporating any structured experience, whether an already-assembled parse tree provided as input or one produced during parsing, into the two taxonomic hierarchies.

L4. Each composite element is added to both hierarchies (its content instance to the content hierarchy and its context instance to the context hierarchy); primitives, which carry no content instance, are added to the context hierarchy alone.

L5. Each addition follows the standard sorting-with-learning procedure of the taxonomic hierarchy that receives it (Cobweb postulates L1–L2).

L3–L5 make recognition and learning two faces of a single elaboration step: if we hand the system a corpus of already-bracketed sentences, it commits their structure straight to memory; if we hand it raw sentences, parsing supplies the same structure. On “the cat saw a dog”, committing *the cat* as an NP routes its content instance to the content hierarchy (recording “DET+N”) and its context instance to the context hierarchy (recording “sentence start, next to *saw*”), so each subsequent step is scored against updated memory. The asymmetric routing of L4 makes the content hierarchy a record of actually-composed structure rather than surface vocabulary. Section 4 shows a concrete system built to these commitments.

4. The TRELLIS System for Chunking and Categorization

TRELLIS instantiates the theory of Section 3 on top of the Cobweb framework of Section 2. We describe the system at the level of design decisions; formulas for the recognition score and threshold appear in Appendix B.

4.1 Representation

Every element carries two attribute–value descriptions. The *content instance* names the parts that compose the element; the *context instance* names its neighbors. Primitives carry only a context instance; composites carry both, as required by R5. Figure 1 illustrates one primitive and one composite drawn from a small parse.

A composite’s content instance names its left and right children by two kinds of attribute: an identifier pointing into the context hierarchy (which says *what* the child is) and a compositional-complexity tag (a small integer recording *how deeply nested* the child’s own subtree is). Both values live in the context hierarchy’s identifier space rather than the surface vocabulary, so the content hierarchy never sees a surface word: every value a content sort encounters was created by a context sort of the corresponding child. The complexity tag lets the content hierarchy split composites whose children share a distributional context but differ in structural depth, such as NP=DET+N versus S=NP+VP, which look identical at the coarsest granularity.

4.2 Organization

TRELLIS organizes long-term memory into two Cobweb taxonomies. The content hierarchy classifies the content instances of composites; the context hierarchy classifies the context instances of every element. The two are linked by a single, well-defined interface: a composite’s content instance is built out of context-hierarchy identifiers (Section 4.1), so the content hierarchy never sees a raw word, and every attribute value it does see was produced by an earlier sort in the context hierarchy. This link is what lets the performance processes unpack an element along both dimensions in a single elaboration step: parsing scores a candidate composite against its content-hierarchy destination (does the composition look consolidated?) and its context-hierarchy destination (do the surroundings look consolidated?) simultaneously, and generation samples a compositional structure

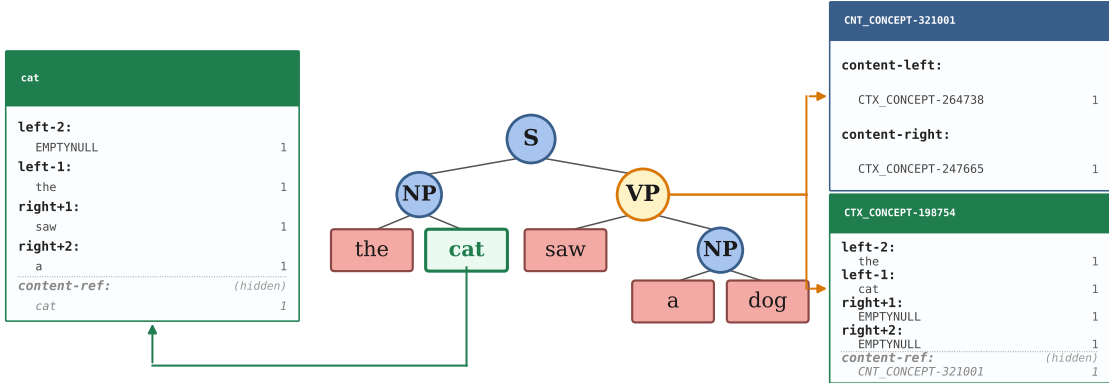


Figure 1. Representation of one primitive (“cat”, green) and one composite (“saw a dog”, amber) from a parse of “the cat saw a dog”. Each box header is the long-term-memory identifier the instance was sorted under. The composite’s content instance holds pointers into the context hierarchy, keeping the two descriptions in complementary formats.

from the content hierarchy while conditioning each child on the context class the structure expects at that slot. Neither direction has to trade the categorical axis against the compositional one, because the interface carries information along both at once. Composites with structurally similar children end up in nearby positions in the content tree, reflecting a shared role (say, as an NP-filling constituent), while a noun and an adjective phrase end up sorted together in the context tree whenever they appear in comparable neighborhoods. Figure 2 shows leaf-level subtrees of each hierarchy and the cross-hierarchy references that stitch them together.

4.3 Performance

TRELLIS realizes P3–P8 as two complementary processes: parsing builds chunks incrementally from an experience, and generation expands chunks back into primitives.

4.3.1 Parsing.

Parsing is a greedy search through the space of possible partonomic parses. It starts with the input’s primitives on a frontier and repeatedly proposes candidate chunks over adjacent pairs, sorting each candidate’s content and context instances through their respective hierarchies. Each sort returns a class-posterior score measuring how well the candidate fits its destination concept relative to the alternatives (Appendix B); the two scores are summed to produce a single recognition score. On “the cat saw a dog”, the first step considers the four adjacent primitive pairs and commits the highest-scoring one as a composite on the frontier.

A candidate is admitted only when its context-hierarchy sort has accumulated enough support: the sort’s path is walked upward from the destination leaf, and if some non-root ancestor’s instance count exceeds the recognition threshold τ , the candidate passes. This climbing-ancestor gate keeps the parser from acting on a leaf that has been visited only once or twice, while still allowing rare-

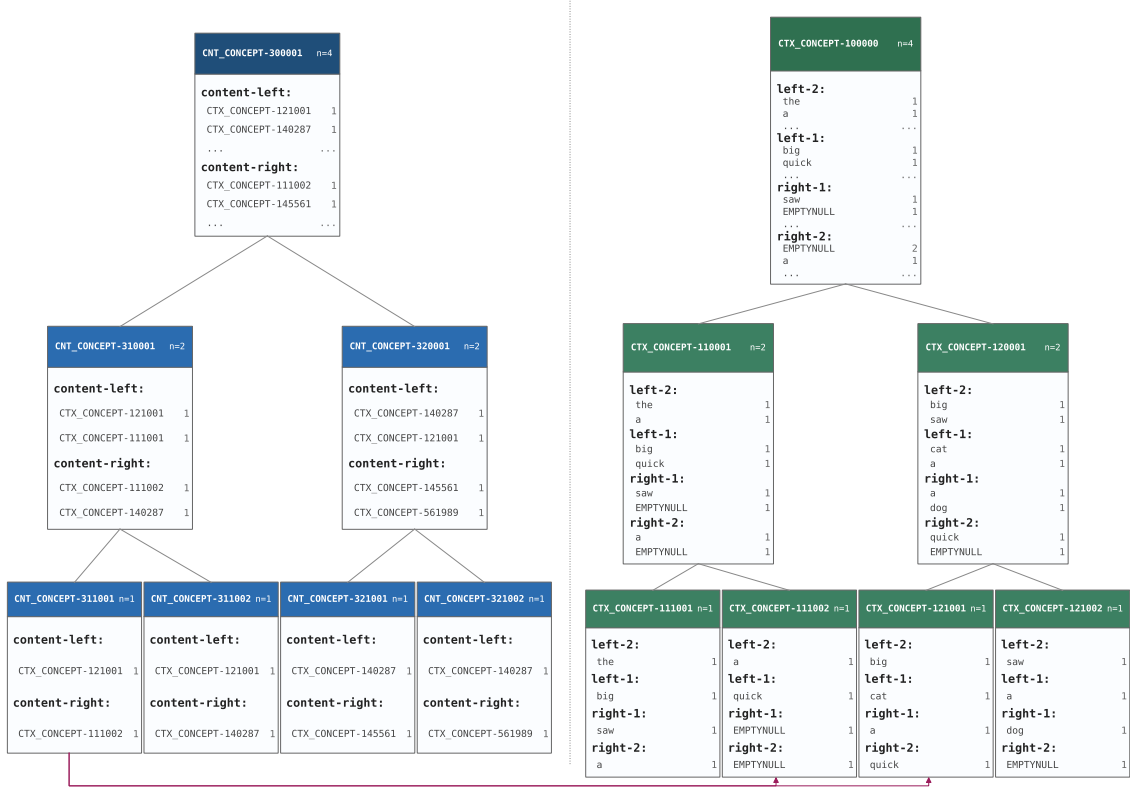


Figure 2. Leaf-level subtrees of the content (left) and context (right) hierarchies. Arrows show the cross-hierarchy link: a content leaf’s child slots name the context concepts that classify its children.

but-consolidated composites to be admitted. Among the admitted candidates, the highest-scoring one is committed to the partonomic parse tree, its two children leave the frontier, and the next step runs against memory updated by the learning procedure of Section 4.4. Parsing terminates when no remaining candidate can be admitted, giving the graceful-failure behavior P5 asks for.

4.3.2 Generation.

Generation runs the parsing loop in reverse. A generation step begins with a seed (an unexpanded composite carrying a content-hierarchy leaf identifier) and expands it into two children by sampling from a *recombination pool* associated with the seed. To build the pool, the system walks up from the seed’s leaf until it reaches an ancestor whose count exceeds τ . This ancestor is the seed’s *basic-level concept* (P6), and the system gathers every committed child pair found under it. The pool therefore draws from a group of pieces the parser has seen often enough to treat as interchangeable at this position, rather than from a single leaf that showed up only once, and sampling from it lets the generator produce new combinations that no single training chunk had supplied.

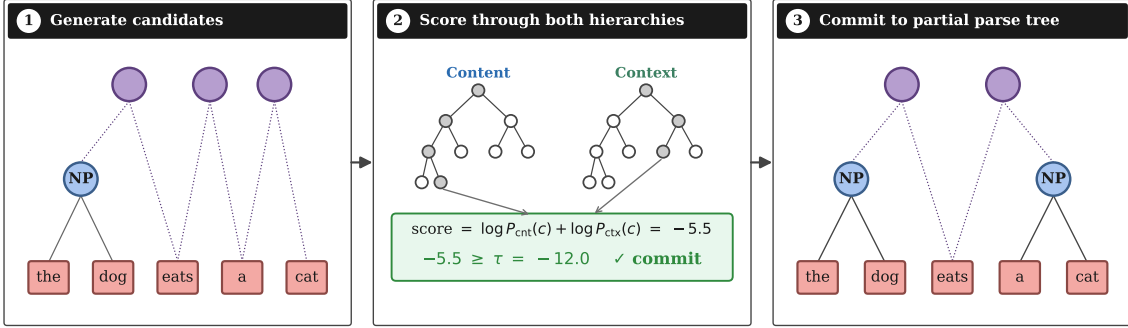


Figure 3. One parsing step: (1) form candidates over adjacent frontier pairs; (2) route each through both hierarchies; (3) commit the best-scoring candidate that clears the recognition threshold.

Each sampled child is then filtered on the context-hierarchy class the parent expects at that slot, so the pool cannot emit a wrong-type constituent (P7). If the filter empties the pool for a slot (typically when a context class has not yet accumulated any compatible chunk), the system falls back to a wider, less-filtered pool that trades a small commission risk for coverage. Generation halts when every leaf of the tree is a primitive.

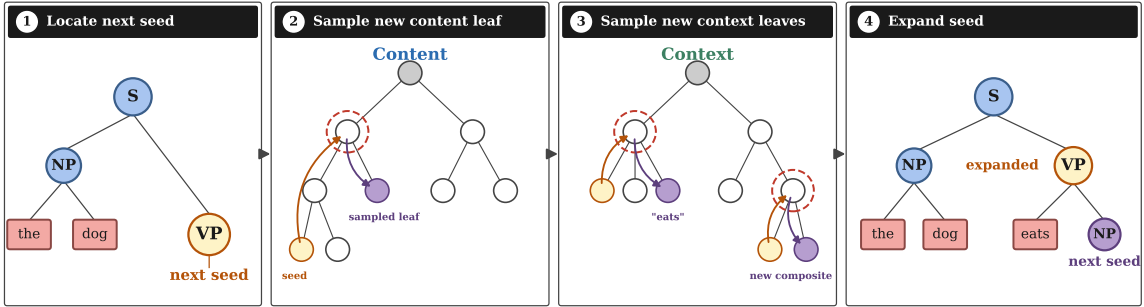


Figure 4. One generation step: (1) pick the next seed; (2) sample a fresh content leaf from a well-supported neighborhood above it; (3) resolve each of the sampled leaf’s two child references against the class the parent expects; (4) attach the results to the seed.

4.4 Learning

Learning in TRELLIS is a simple adaptation of the existing learning protocol of Cobweb. The system introduces no new learning mechanism: each committed composite and each frontier primitive is handed to Cobweb’s ordinary sort-and-learn cycle in whichever hierarchies apply. Composites go into both hierarchies (their content instance to the content hierarchy, their context instance to the context hierarchy); primitives go only into the context hierarchy, since they carry no content instance. Because both sorts happen immediately after the commit, candidates produced later in the same parse are scored against a memory that already reflects the chunks committed earlier. There is

no separate “chunk creation” subroutine: every committed composite becomes a fresh node in one or both hierarchies, and that node is exactly what a later parsing step uses as a category label the next time a structurally similar chunk appears.

5. Evaluation of TRELLIS

The evaluation asks whether TRELLIS does what the postulates promise: recognize gold-standard chunks in inputs it has not seen and generate new outputs consistent with the grammar it has learned. Following Langley (1996)’s structure for evaluating incremental learners, we identify the dependent variables, the independent axes our synthetic grammars vary, and the questions the study is meant to answer.

Dependent variables. We follow Langley & Stromsten (2000)’s framing, which decomposes errors into two categories: an *omission* is gold structure the learner failed to produce (a false negative), and a *commission* is structure it produced that was not in the gold data (a false positive). On the parsing side we report *bracket accuracy* on held-out gold parses, which falls when a parse either misses a gold bracket (omission) or asserts one the gold parse lacks (commission). On the generation side we report the *grammaticality rate*, the fraction of sampled sentences the source grammar accepts; this captures commission directly, while generation-side omission is not well-defined, since the generator samples freely rather than enumerating the grammar.

Independent variables. Two axes, matched to the two dimensions our theory keeps separate. First, the number of non-terminal categories, holding the lexicon fixed; each new non-terminal is a chunk type the content hierarchy must distinguish. Second, the number of terminals per part-of-speech class, holding productions fixed; thinning per-word support pushes the context hierarchy to generalize across rare surface forms.

Questions. We ask four questions through these experiments: whether the parser recovers gold brackets on held-out sentences (Q1); whether the generator stays inside the source grammar (Q2); how each ability scales as we add non-terminal categories (Q3); and how each ability scales as we add terminals per part-of-speech class (Q4).

5.1 Setup

The categorical axis uses three strictly nested grammars of progressively richer constituent structure ($\text{SMALL} \subset \text{MED} \subset \text{LARGE}$); the distributional axis holds the MED productions fixed and varies the lexicon per part-of-speech class across LOW (11 terminals), MED (22), and HIGH (39). Full productions and lexicons are in Appendix A. Each condition drew a 400-sentence derivation-annotated corpus, trained TRELLIS incrementally on the first 320 sentences, and evaluated on the 80-sentence held-out remainder. All metrics are five-seed means with a $\pm 1\sigma$ band. A single parameter set (the Cobweb uniform-prior smoothing α and the recognition threshold τ) is shared across every condition; we deliberately avoid per-corpus tuning because a system that requires re-tuning at every categorical or distributional variation is not a system that has learned to categorize and chunk in the sense the postulates require.

5.2 Scaling Behavior Across Grammar Complexity

Figure 5 plots parse accuracy on the left and generation grammaticality on the right across the three grammars as the training corpus grows. Parse accuracy orders monotonically by grammatical complexity, converging to 1.00 on SMALL, 0.97 on MED, and 0.94 on LARGE: each new non-terminal is a chunk type the content hierarchy must eventually distinguish, so more categories means more restructuring events before the parser settles, and both omission and commission stay near-zero at convergence. Generation grammaticality reaches 0.98–1.00 on every grammar; the small residual commission on MED and LARGE traces to child slots whose recombination pools (the eligible chunks sampled from at each generation step) do not yet contain a context-compatible chunk, so the generator falls back to a wider, less filtered pool that can occasionally admit a wrong-type constituent.

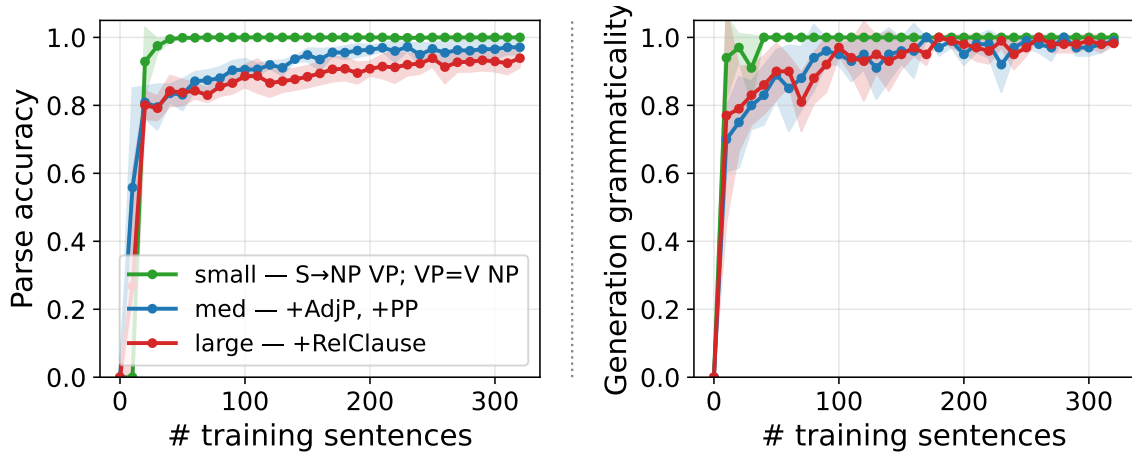


Figure 5. Parse accuracy (left) and generation grammaticality (right) across the three grammars (5 seeds, $\pm 1\sigma$ band). Both curves are omission/commission avoidance rates: higher is better.

5.3 Scaling Behavior Across Terminal Vocabulary Size

Figure 6 repeats the overlay across the three lexicon sizes on the MED grammar. The two-hierarchy organization absorbs per-terminal sparsity by construction: the context hierarchy groups distributional equivalents (nouns with nouns, verbs with verbs) so each terminal inherits part-of-speech support from its siblings, and the content hierarchy sees no growth in its attribute vocabulary since composites still refer to children by context-hierarchy identifier. Parse accuracy stays close to the MED-grammar level across all three sizes (0.95, 0.96, 0.89 for LOW, MED, HIGH). Generation grammaticality converges around 0.96 on MED and HIGH and 0.93 on LOW, where the smaller vocabulary yields fewer distinct chunks per context class and the wrong-type fallback fires slightly more often.

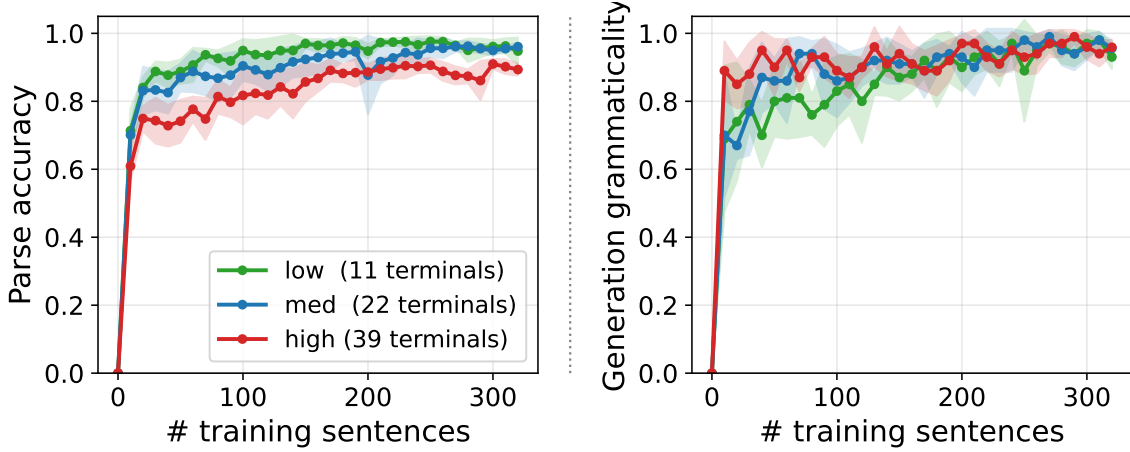


Figure 6. Parse accuracy (left) and generation grammaticality (right) across the three lexicon sizes on the MED grammar (5 seeds, $\pm 1\sigma$ band, omission/commission avoidance).

5.4 Summary

The parser recovers gold brackets at accuracy between 0.89 and 1.00 across every condition, an omission budget of at most 0.11 (Q1). The generator’s commission rate stays under 0.07 everywhere, producing grammatical outputs at rates between 0.93 and 1.00 (Q2). Both abilities degrade gracefully as non-terminals are added, with parsing degrading slightly faster than generation (Q3), and both are largely insensitive to lexicon size (Q4). What the two experiments together show is why the concept-formation and chunking literatures need each other. Our chunks are defined over context-hierarchy concept classes rather than surface tokens, so the categorical generalization the concept side supplies is what keeps a chunk recognizable as we vary the terminals and non-terminals underneath it. Concepts without chunks leave compositional structure out of reach; chunks without concepts collapse the moment a familiar configuration appears with unfamiliar components. The robustness on both axes falls out only when the two are unified.

6. Related Work

Several bodies of work cover some but not all of what TRELLIS delivers: unified accounts of categorical and compositional memory, incremental parsing and inverse generation under a single learning loop, discrete role–filler binding, and unsupervised acquisition of constituent structure.

The closest precursor is Langley (2025)’s essay arguing that a unified theory should treat concept-hood and chunk-hood as two aspects of every representation. That essay proposes commitments any such system would need to satisfy but does not commit to an implementation; our theory is one concrete instance of it. The Matsakis system (Matsakis et al., 2023) extends Cobweb with iterative additions but does not draw a content–context distinction. The convolutional Cobweb variant (Lian & MacLellan, 2024) pairs a chunking-style construction process with a classification-style hierarchy but fixes the construction by architecture and operates on images rather than the sequential parto-

nomic structure that drives our parsing loop. The CHREST family (Gobet & Lane, 2012; Richman, 1991), originally built to model chess-expert memory, organizes chunks in a discrimination network: an internal decision tree whose nodes acquire test-based subclassifications as new chunks are recognized. This machinery captures acquisition and rapid recognition well, but its chunks match input by exact lookup rather than by generalizing across structurally similar instances, which is exactly the limitation the two-hierarchy organization is designed to lift.

Adjacent to the cognitive-systems tradition, the connectionist literature asks what a compositional representation must support. Smolensky (1990) developed *tensor-product representations*, in which a role (say “subject”) and a filler (say “the cat”) are bound together as the outer product of their vectors, so a structured representation is a sum of role-filler bindings recoverable later. Plate (1995)’s *holographic reduced representations* obtain a similar effect with circular convolution as a compressed binding operator. Our content/context split reads as a discrete counterpart: two indexed descriptions keep role and filler from interfering while remaining ordinary Cobweb instances that need no vector algebra.

The problem of inducing a grammar from unannotated sentences asks for the same combination of compositional structure and categorical generalization our theory addresses. Wolff (1982) introduced an early system that merges frequent adjacent pairs and generalizes the resulting categories; its two operations prefigure our two hierarchies, though they are not maintained as separate structures. Langley & Stromsten (2000) introduced GRIDS, which incrementally hypothesizes grammar productions and evaluates candidate grammars by a minimum-description-length (MDL) criterion that trades the size of the grammar against how compactly it encodes the training corpus. TRELIS borrows the omission/commission decomposition from that same paper. These systems treat the categorical and compositional aspects as a single optimization rather than as two dimensions of memory, and correspondingly lack a single substrate from which both parsing and generation fall out under one elaboration loop.

A third body of work asks whether local context alone can drive syntactic categorization. Distributional embeddings cluster words that occur in similar contexts, and probes over contextualized models recover part-of-speech tags and constituency structure from intermediate layers. Against the concepts–chunks dichotomy, these models sit closer to concept formation: they extract a context-driven categorical signal much like the one our context hierarchy maintains. They are further from chunking: constituent structure is present latently but not laid out as a manipulable partonomy, and behavioral tests on systematic generalization continue to expose compositional failures. Our theory pairs the context-driven signal with an equally explicit content-driven one, so composition is a first-class property of memory.

7. Concluding Remarks

We have presented a postulate-driven theory that combines the categorical commitments of concept formation with the compositional commitments of chunking: every element carries a content and a context description, organized across two Cobweb hierarchies, with one elaboration loop turning them into parsing, generation, and incremental learning. TRELIS recovers gold chunks at high precision and recall and generates near-perfectly grammatical sentences from memory. One caveat

to the implementation is that the basic-level concept is itself an active area of study; our count-based maturity cut identifies the well-supported level of abstraction heuristically rather than against an independently verified definition, and future work on principled basic-level identification would replace this cut directly.

The unified theory places chunks and contexts in a mutual dependence neither can do without: chunks are identifiable only because their contexts are similar, and contexts are informative only because they constrain the chunks that fill them. This dependence is where the empirical asymmetry between parsing and generation finds its natural explanation: parsing fails when context-hierarchy support for a candidate has not yet cleared the recognition threshold, while generation succeeds because every chunk in the content hierarchy was placed there by a parse that had already cleared it. The two procedures are inverses of each other but draw on the two hierarchies asymmetrically, and the asymmetry is what gives the system its graceful-failure behavior under threshold tuning.

Three future endeavors follow. *Unsupervised chunk learning* is the most pressing: TRELLIS still relies on a tuned recognition threshold τ , and earlier statistical approaches (MDL grammars, mutual-information chunkers that merge adjacent tokens whenever their co-occurrence exceeds a cutoff) struggled to say what makes a chunk *good* once superficial recurrence had been exhausted. The theory points toward a chunk-intrinsic answer: a chunk is good to the extent that it is stable across contexts, reusable as a part of larger chunks, and predictive of its neighbors. An adaptive criterion tracking memory growth should identify such chunks without manual tuning. The second endeavor is to push the substrate *beyond sentences*: upward into paragraphs and discourse units, downward into letters and phonemes, with the basic-level frontier shifting as more abstract chunks accumulate; the single left/right relation used here should likewise generalize to broader discrete-relation vocabularies (gridlike neighborhoods, compass directions, temporal sequences) without changing the postulates. The third is representational: content and context are held in separate hierarchies here for clarity, but the compositional nature of the domain invites a more intertwined view in which the two lenses inform each other within a single sort. The unification is one the cognitive-systems literature has gestured toward for decades but rarely committed to in implementation, and the theory together with TRELLIS establishes it as a viable foundation for further work.

Acknowledgements

We would like to thank Chris MacLellan, Zekun Wang, Xin Lian, Kyle Moore, and friends at the Teachable AI Lab for discussions.

References

- Chase, W. G., & Simon, H. A. (1973). Perception in chess. *Cognitive Psychology*, 4, 55–81.
- Fisher, D. H. (1987). Knowledge acquisition via incremental conceptual clustering. *Machine Learning*, 2, 139–172.
- Gennari, J. H., Langley, P., & Fisher, D. H. (1989). Models of incremental concept formation. In J. G. Carbonell (Ed.), *Machine learning: Paradigms and methods*, 11–61. Cambridge, MA: MIT Press.

- Gobet, F., & Lane, P. C. R. (2012). CHREST: A model of perception, learning, and problem solving. In N. M. Seel (Ed.), *Encyclopedia of the sciences of learning*, 556–559. New York, NY: Springer.
- Hampton, J. A. (1988). Overextension of conjunctive concepts: Evidence for a unitary model of concept typicality and class inclusion. *Journal of Experimental Psychology: Learning, Memory, and Cognition*, 14, 12–32.
- Langley, P. (1996). *Elements of machine learning*. San Francisco, CA: Morgan Kaufmann.
- Langley, P. (2025). Concepts and chunks in cognitive systems. *Advances in Cognitive Systems*, 11, 1–12.
- Langley, P., & Stromsten, S. (2000). Learning context-free grammars with a simplicity bias. *Proceedings of the Eleventh European Conference on Machine Learning* (pp. 220–228). Barcelona, Spain: Springer.
- Lian, X., & MacLellan, C. J. (2024). Convolutional Cobweb: A new approach to image classification through concept formation. *Advances in Cognitive Systems* (pp. 221–238).
- MacLellan, C. J., Harpstead, E., Aleven, V., & Koedinger, K. R. (2016). TRESTLE: A model of concept formation in structured domains. *Advances in Cognitive Systems* (pp. 131–150).
- MacLellan, C. J., Matsakis, N., & Langley, P. (2022). Efficient induction of language models via probabilistic concept formation. *Advances in Cognitive Systems* (pp. 1–16).
- Matsakis, N., MacLellan, C. J., & Langley, P. (2023). Incremental induction of phrase-structure grammars through probabilistic concept formation. *Proceedings of the Eleventh Annual Conference on Advances in Cognitive Systems*. Atlanta, GA.
- Miller, G. A. (1956). The magical number seven, plus or minus two: Some limits on our capacity for processing information. *Psychological Review*, 63, 81–97.
- Murphy, G. L. (2002). *The big book of concepts*. Cambridge, MA: MIT Press.
- Murphy, G. L., & Medin, D. L. (1985). The role of theories in conceptual coherence. In *Psychological review*, volume 92, 289–316.
- Osherson, D. N., & Smith, E. E. (1981). On the adequacy of prototype theory as a theory of concepts. *Cognition*, 9, 35–58.
- Plate, T. A. (1995). Holographic reduced representations. *IEEE Transactions on Neural Networks*, 6, 623–641.
- Richman, H. B. (1991). *Discrimination net models of expertise*. Doctoral dissertation, Department of Psychology, Carnegie Mellon University, Pittsburgh, PA.
- Rosch, E., Mervis, C. B., Gray, W. D., Johnson, D. M., & Boyes-Braem, P. (1976). Basic objects in natural categories. *Cognitive Psychology*, 8, 382–439.
- Smolensky, P. (1990). Tensor product variable binding and the representation of symbolic structures in connectionist systems. *Artificial Intelligence*, 46, 159–216.
- Wolff, J. G. (1982). Language acquisition, data compression, and generalization. *Language and Communication* (pp. 57–89).

Appendix A. The Synthetic Grammars

This appendix lists the productions and terminal lexicons of the three synthetic context-free grammars used in Section 5. Every non-terminal expansion has one or two right-hand-side elements, matching the strictly binary representation used by the parser, and sentences are derived by selecting uniformly at random from the right-hand-side alternatives. The three grammars are strictly nested: every production and terminal of **SMALL** is retained in **MED**, and every production and terminal of **MED** is retained in **LARGE**.

SMALL. A determiner-noun NP and a transitive VP, sharing **MED**’s full determiner, noun, and verb lexicons (only the Adj and P classes and the modifier productions are absent).

S	→ NP VP
NP	→ Det N
VP	→ V NP
Det	→ the a
N	→ cat dog man woman park telescope
V	→ saw liked chased found admired

MED. Extends **SMALL** with adjective recursion through an **AdjP** head that always carries at least one adjective, prepositional phrases, and a packaged direct object plus PP through the intermediate **VPobj**; it introduces the Adj and P classes (additions over **SMALL** in **bold**). Bare-noun NPs use the no-modifier shortcut “NP → Det N” so the grammar never produces a unary syntactic node.

S	→ NP VP
NP	→ Det N Det AdjP
AdjP	→ Adj N Adj AdjP
VP	→ V NP V VPobj
VPobj	→ NP PP
PP	→ P NP
Det	→ the a
N	→ cat dog man woman park telescope
Adj	→ big small red quick lazy
V	→ saw liked chased found admired
P	→ with in on under

LARGE. Adds relative clauses on top of **MED**: a relative-clause NP shape (Det Nbar), the Nbar and RelClause non-terminals, and the RelPro part-of-speech class, together with a handful of extra terminals (additions over **MED** in **bold**). Splitting **AdjP** (adjective-only) from Nbar (head with relative clause) keeps each label semantically faithful; recursion is weighted toward the non-recursive alternatives so RelClause sentences do not dominate the corpus.

S	→ NP VP
NP	→ Det N Det AdjP Det Nbar
AdjP	→ Adj N Adj AdjP
Nbar	→ N RelClause AdjP RelClause
VP	→ V NP V VPobj
VPobj	→ NP PP
RelClause	→ RelPro VP
PP	→ P NP
Det	→ the a this that
N	→ cat dog man woman park telescope boy girl teacher
Adj	→ big small red quick lazy tall curious
RelPro	→ who that which
V	→ saw liked chased found admired carried read
P	→ with in on under near

Terminal-experiment lexicons. The terminal-vocabulary experiment (Section 5.3) holds the MED productions fixed and varies only the lexicon, drawing the first k words of a shared pool for each part-of-speech class. The three variants are strictly nested ($\text{LOW} \subset \text{MED} \subset \text{HIGH}$):

Part of speech	LOW (11)	MED (22)	HIGH (39)
Det	the, a	the, a	the, a, this, that
N	cat, dog, man	cat, dog, man, woman, park, telescope	cat, dog, man, woman, park, telescope, robot, apple, book, girl, boy, bird
Adj	big, small	big, small, red, quick, lazy	big, small, red, quick, lazy, ancient, blue, tall, wise
V	saw, liked	saw, liked, chased, found, admired	saw, liked, chased, found, admired, carried, read, knew
P	with, in	with, in, on, under	with, in, on, under, near, behind

Appendix B. Implementation Formulas

Recognition score. The Cobweb log-probability of an instance $\mathbf{i}$ under a concept C is, by R3,

$$\log P(\mathbf{i} \mid C) = \sum_{(a,v) \in \mathbf{i}} \log P(v \mid C, a), \quad P(v \mid C, a) = \frac{n_{C,a,v} + \alpha_a}{n_{C,a} + V_a \alpha_a},$$

where $n_{C,a,v}$ is the count of value v for attribute a at concept C , V_a is the value-vocabulary size, and α_a is the uniform-prior smoothing (R3). Candidates are ranked not by this raw log-probability but by the *class posterior* it induces: sorting instance $\mathbf{i}$ into a hierarchy selects some class C , and the score is $\log P(C \mid \mathbf{i}) = \log P(\mathbf{i} \mid C) + \log P(C) - \log P(\mathbf{i})$. Normalizing out the instance

marginal makes the score discriminative, so a non-constituent that fits nowhere in particular scores low. Content and context scores are the two posteriors s_c and s_x ; the ranking key is $s_x + s_c$, with s_x breaking ties.

Recognition threshold (climbing ancestor). An element’s path in the context hierarchy is walked upward from its destination leaf. The candidate is admitted when a non-root ancestor has accumulated an instance count exceeding the recognition threshold τ ; otherwise it fails. Sharing this gate between primitives and composites is what gives the parsing loop a single notion of “mature representation” across both layers.

Generation anchor and chunk replay. The same climbing-ancestor notion realizes the level P6 samples from: the anchor above a leaf is the shallowest ancestor whose count exceeds τ . During training every committed chunk is recorded as a tuple (parent-leaf, left-child, right-child), indexed by both its parent-leaf identifier and that leaf’s maturity-cut ancestor. Generation samples a sentence-root chunk as the seed and recurses on each child: primitives are emitted directly; composites are expanded by sampling a chunk from the maturity-level pool, restricted to those whose own context class matches what the parent expects at that slot (P7 conditioning). The maturity-level pool makes recombinations no single leaf could produce reachable, while context conditioning keeps the wider pool from emitting a wrong-type constituent.

Content-instance encoding. Each content attribute value is a small bag over concept identifiers rather than a single symbol, preserving the concept’s own uncertainty rather than forcing a commit to its most-likely value. A visible complexity tag on each child lets category utility separate composites whose children share a distributional context but differ in structural depth (NP=DET+N versus S=NP+VP, which share the same right-hand-side signature at the coarsest granularity). Because the context hierarchy restructures as new instances are absorbed, content references can dangle over time; TRELLIS canonicalizes each reference to a recent ancestor of its target leaf and rewrites stored instances whenever the context tree restructures.